

Reflection Report: RAG-Assisted Prompt Iteration for Code Refactoring

GitHub repository link : <https://github.com/nuzaeromar/AI-Augmented-SE.git>

Final Prompt:

The final prompt that worked for my case:

```
You are a software engineer refactoring Python code.

## Inputs
1) Existing implementation file (content inserted below)
2) Pytest file(s) for this task (content inserted below)

## Goal
1) Refactor the implementation to improve readability and maintainability while preserving behavior exactly as validated by the provided tests.
2) Specify exact function signatures and return types.
3) Clarify precision requirements for floating-point results.
4) Keep the logic, input and output for all the functions of original and refactored same
5) Review all test cases and ensure all required functions are properly implemented with correct logic
6) Keep the equation of any mathematical function similar to the original code in the refactored code

## Strict Rules to Follow
1) Make sure all the function and variable names remain same
2) If the function is in a different module, ensure proper imports are in place
3) Ensure all implementation files exist in the `tasks/` directory with the exact names referenced in the tests

## Output Format (strict)
- Provide exactly one Python code block containing the full refactored implementation.
- After the code block, provide the checklist in 5 to 10 bullets.
- Do NOT include any additional text.

---

## Implementation file content
<<<IMPLEMENTATION>>>
```

Short Reflection

Team Members

- Nuzaer Omar (me)
- Mizanur Rahman Jewel

Overview

In this assignment, we refactored Python programs using prompt iteration guided by a Retrieval-Augmented Generation (RAG) pipeline. The goal was to preserve program behavior while improving readability and maintainability, and to use retrieval to systematically diagnose test failures and regressions. We worked with a repository containing 40 Python tasks and their corresponding pytest files, using FAISS-based retrieval over prior failures, test expectations, and original task descriptions to iteratively refine our prompts.

Failure Patterns Observed

Most failures fell into two dominant categories: (1) `NameError`/API mismatch caused by renaming or not preserving the required function signature, and (2) semantic drift, where the refactor used a reasonable but different interpretation (wrong formula, wrong indexing, wrong return convention like `False` vs `None`).

1. **API / Function-name mismatch (`NameError` failures):** The refactored code did not expose the exact function name that the tests call. Even when the logic existed, it was renamed, wrapped, or placed under a different identifier, like: pytest raised `NameError`.

Typical root causes we observed:

- The model changed names (snake_case, shorter names) or corrected spelling/casing as per their usual memory cases.
 - The model moved logic into helper functions and forgot to keep the original public function name.
 - The model changed capitalization patterns (sum_of_Factors vs sum_of_factors) or underscores.
2. **Semantic drift despite syntactic correctness:** The function ran and returned a value, but the refactor changed the meaning of the computation often by choosing a reasonable interpretation that didn't match the benchmark/test's intended one. Examples include:
 - `kth_element` (task_101): returned **5** instead of **3** (algorithm/definition mismatch for "kth element").
 - `angle_complex` (task_124): returned π instead of $\pi/2$ (wrong angle convention / quadrant handling).

- `parallelogram_perimeter` (task_882): returned **60** instead of **400** (formula mismatch). Refactor used standard perimeter $2(a+b)$ while test expected something else.
3. **Incorrect return conventions:** The function returned a logically plausible value, but not the exact value/type expected by the tests (especially `None` vs `False`, or very specific numeric results). Examples seen in:
 - `common_element` returned `False` instead of `None` when no common element was found.
 4. Over-generalized logic: Functions such as `upper_ctr` (task_461) counted all uppercase characters, whereas the test expected a more constrained interpretation (e.g., counting uppercase letters excluding the first character or ignoring specific positions).

What the RAG Chat Retrieved That Was Useful

The RAG chat was effective because it retrieved `pytest` assertions, original task context, and function signatures, which together defined the true behavioral specification of each task. Many early failures were caused by missing or renamed functions (`NameError`), and RAG helped identify the exact public function names expected by the tests, allowing us to eliminate a large number of errors.

RAG also surfaced hidden behavioral contracts embedded in tests, such as exact return values, return types, and edge-case handling (e.g., returning `None` instead of `False`). This evidence prevented us from relying on intuitive interpretations and instead align the prompt with their consecutive expectations, where tests act as the ground truth.

By incorporating these retrieved constraints into a more general prompt we were able to preserve function signatures, return types, and logic exactly. We were able to remove about 13 errors, leaving only a small number of remaining failures caused by deeper semantic interpretation differences rather than missing APIs or contracts.

How Retrieved Evidence Changed the Prompt

The evidence retrieved through the RAG chat showed that most failures were caused by violations of test-defined contracts rather than syntax errors. In particular, retrieved `pytest` files revealed that tests rely on exact function names, exact return types, and specific mathematical formulas, and that even small deviations (renaming functions, changing equations, or adjusting return conventions) resulted in failures.

Based on this evidence, the prompt was revised to explicitly enforce behavioral invariance: preserving function signatures, variable names, input–output behavior, and original mathematical equations exactly as validated by the tests. Retrieved failures such as repeated `NameError` led to strict rules about keeping function names unchanged and ensuring all required files exist in the correct modules. Similarly, discrepancies in numeric outputs (e.g., angle and geometry functions)

motivated the addition of precision constraints and a rule to keep mathematical equations identical to the original implementation.

Overall, the retrieved evidence shifted the prompt from a general clean refactoring objective to a test-driven, contract-preserving refactoring strategy, ensuring improvements in readability without introducing semantic drift.

Remaining Errors and Insights, and Successful Refactorings

After applying RAG-guided prompt refinement, we successfully removed the majority of failures (from the initial 17 to 4 errors), completing 36 out of 40 refactorings successfully. Most early errors were due to missing or renamed functions and were resolved once the prompt was updated to strictly preserve function names, signatures, and file structure. The remaining failures are concentrated in a small set of tasks where the refactored implementations violated implicit behavioral contracts encoded in the unit tests. These are semantic mismatches rather than implementation or API-level errors.

For Task 101 (`kth_element`), the refactored code returns 5 instead of the expected 3, indicating a mismatch in how the “kth element” is interpreted. The function appears to use a different indexing or ordering convention than the test expects.

In Task 124 (`angle_complex`), the function returns π instead of $\pi/2$ for the input (0, 1j). This suggests that the refactoring used a mathematically reasonable but incorrect angle computation rule, such as improper handling of quadrants or argument order in trigonometric functions.

For Task 431 (`common_element`), the function returns False when no common element exists, whereas the test expects None. This reflects a non-standard return convention enforced by the benchmark.

For Task 461 (`upper_ctr`), the refactored code returns a count of 2 instead of the expected 1. This indicates that the refactoring counted uppercase characters using a broader or more general rule than what the test enforces. The test case reflects a different, more restrictive counting convention for uppercase letters, which may exclude certain characters or positions (e.g., ignoring the first character or applying a specific condition).

Ensuring Prompt Generalizability

To keep the final prompt generalizable and not task specific, we:

- Avoided hard-coding rules tied to individual functions.
- Framed constraints in terms of test-driven behavior preservation, applicable to any task.
- Used abstract principles such as, “Keep the logic, input, and output for all functions of the original and refactored same.”
- Ensured the prompt could be reused across new repositories with similar test-based specifications.

This resulted in a reusable, scalable prompt suitable for future RAG-assisted debugging and refactoring tasks.

Conclusion

This assignment demonstrated that prompt engineering alone is insufficient for reliable refactoring at scale. However, when combined with retrieval over tests, failures, and prior iterations, RAG provides a powerful mechanism for diagnosing regressions and systematically improving prompts.